

ESP32 firmware — MQTT migration guide

Everything the tracker unit has to do, and how to build it.

Read `HARDWARE.md` for the contract this must satisfy. This document is the implementation guide: what changes, the exact bytes on the wire, working code for the hard parts, and how to prove it works.

`HARDWARE_BUILD.md` covers everything that is not code — power, antennas, SIM, provisioning and fitting. If a unit resets when it transmits, or gets no GPS fix in a bus, the answer is there rather than here.

1. What is changing, and what is not

The unit used to POST a signed JSON body over HTTP to a relay server. That relay and the VM it ran on are gone. The unit now publishes the same signed body to a public MQTT broker, which the Raspberry Pi subscribes to.

Unchanged	Changed
The JSON body and every field name	Transport: HTTP POST becomes MQTT PUBLISH
HMAC-SHA256 signing, same algorithm	Signature moves from the X-Sig header into the payload
Monotonic counter, same rules	Endpoint: now <code>broker.emqx.io:1883</code>
5 second interval	New: register an MQTT Last Will
Metres per second, <code>hdg: null</code> when unknown	New: offline fixes are dropped, never buffered
No TLS, no certificates	

The signing logic you already have is correct and does not need rewriting. What you are adding is an MQTT client and a different way of framing the message.

Why the change

Nothing in the system can accept an incoming connection any more. The buses are behind carrier NAT on 2G, and the Raspberry Pi now runs on a phone hotspot, so there is no router to forward a port on and no fixed public address anywhere. A public broker is a meeting point that everything can dial *out* to.

It has to be a plaintext public broker rather than a managed service because the SIM900A has no TLS, and every managed free tier requires it.

2. Target

Setting	Value
Broker host	<code>broker.emqx.io</code>
Port	1883

Setting	Value
Transport	plain TCP, no TLS
Protocol	MQTT 3.1.1
Username / password	none
Clean session	yes
Keepalive	60 seconds
Client id	cbt7f3c9e21b-<deviceid>, e.g. cbt7f3c9e21b-esp32-01

The topic root is `cbt7f3c9e21b`. It is deliberately meaningless: this is a shared public broker with thousands of users, and a name like `campus/bus` would collide with someone else's traffic. It must match the server exactly.

The client id is prefixed with the same root for the same reason — see §9.

Topic	Payload	QoS	Retain
<code>cbt7f3c9e21b/bus/<busid>/gps</code>	signature, dot, signed JSON	0	no
<code>cbt7f3c9e21b/bus/<busid>/status</code>	online on connect	1	yes
<code>cbt7f3c9e21b/bus/<busid>/status</code>	offline as the Last Will	1	yes

`<busid>` is this unit's bus: `bus1`, `bus2`, `bus3`, `bus4`, `ac_mbh`, `ac_lh`.

Publishing on another bus's topic is rejected even if the signature is valid.

3. The message

```
d0d9d4c131e5b25c618be70720948ae790b564804543d6f8a7
43a88716567e39.{ "b": "bus1", "d": "esp32-01", "c": 1234,
"lat": 11.3185, "lng": 75.9379, "spd": 6.4, "hdg": 312 }
```

(one line on the wire -- wrapped here only to fit the page)

Exactly: **64 hex characters, one dot, then the JSON bytes you signed.**

MQTT has no header to carry a signature, which is why it is inline. The server splits on the first dot and verifies the remainder — it never re-parses and re-serialises, so the bytes it checks are the bytes you sent.

That means the rule from the old spec still holds, and is now the single most important thing in this document:

Build the JSON once into a buffer, sign that buffer, publish that same buffer.

Rebuilding it between signing and publishing changes the bytes, and the signature will not match.

4. Which implementation route

There are two ways to get MQTT onto this hardware.

Route A: PPP plus esp-mqtt — recommended

Put the modem into PPP data mode so the ESP32 owns the TCP/IP stack, then use the `esp-mqtt` component with a plain `mqtt://broker.emqx.io:1883` URI. ESP-IDF handles packet construction, keepalive, reconnection and the Last Will.

You write almost no protocol code. Use `esp_modem` with the SIM800 device profile — the SIM900A responds to the same command set for PPP.

Verify early that PPP comes up on your exact module and firmware revision. If it does, take this route.

Route B: hand-rolled MQTT over AT commands

If PPP will not come up, build the MQTT packets yourself and push them through the modem's TCP socket with `AT+CIPSTART` and `AT+CIPSEND`. You need three packet types. Section 6 gives the bytes.

This is perhaps 100 lines and entirely doable, but you own the framing, keepalive timing and reconnection.

5. Program structure

```
boot:
  read counter seed          (section 8)
  start GPS
  modem: attach GPRS

connect:
  open TCP to broker.emqx.io:1883
  send CONNECT (with Last Will)
  wait for CONNACK, check return code 0
  publish "online" to status topic, QoS 1, retained

loop every 5 s:
  read GPS fix
  if no valid fix: skip this cycle
  build body -> sign -> publish "<sig>.<body>" to gps topic, QoS 0
  if socket dead: go to connect

every 60 s with no other traffic:
  send PINGREQ, expect PINGRESP
  no PINGRESP within ~10 s -> treat the socket as dead, reconnect
```

Never block the GPS loop on a network call. A publish that cannot go out is discarded, not queued.

6. MQTT packet construction (Route B only)

Skip this section entirely if you took Route A.

Remaining length

Every MQTT packet is a one-byte type, then a variable-length integer giving the length of everything after it, then the body. Seven bits per byte, high bit set means another byte follows.

```
// Writes 1-4 bytes. Returns how many were written.
int mqtt_len(uint8_t *out, uint32_t len) {
    int n = 0;
    do {
        uint8_t b = len % 128;
        len /= 128;
        if (len > 0) b |= 0x80;
        out[n++] = b;
    } while (len > 0);
    return n;
}
```

Strings inside packets are a two-byte big-endian length followed by the bytes, with no terminator.

```
int mqtt_str(uint8_t *out, const char *s) {
    uint16_t n = strlen(s);
    out[0] = n >> 8;
    out[1] = n & 0xFF;
    memcpy(out + 2, s, n);
    return n + 2;
}
```

CONNECT

```
// clean session 0x02 | will 0x04 | will QoS1 0x08 | will retain 0x20
#define CONNECT_FLAGS 0x2E

int build_connect(uint8_t *buf, const char *client_id,
                  const char *will_topic, const char *will_msg) {
    uint8_t var[512];
    int v = 0;
    v += mqtt_str(var + v, "MQTT");           // protocol name
    var[v++] = 0x04;                          // protocol level = 3.1.1
    var[v++] = CONNECT_FLAGS;
    var[v++] = 0x00;                          // keepalive high byte
    var[v++] = 60;                            // keepalive low byte, seconds
    v += mqtt_str(var + v, client_id);
    v += mqtt_str(var + v, will_topic);
    v += mqtt_str(var + v, will_msg);         // "offline"

    int n = 0;
    buf[n++] = 0x10;                          // CONNECT
    n += mqtt_len(buf + n, v);
    memcpy(buf + n, var, v);
    return n + v;
}
```

The broker replies with a four-byte CONNACK, the last byte being the return code. **Return code 0 means accepted.** Anything else and you must not continue; log the code and retry after a backoff.

PUBLISH, QoS 0

```
int build_publish(uint8_t *buf, const char *topic,
                 const char *payload, bool retain) {
    uint16_t plen = strlen(payload);
    uint8_t var[1024];
    int v = mqtt_str(var, topic);    // no packet id at QoS 0
    memcpy(var + v, payload, plen);
    v += plen;

    int n = 0;
    buf[n++] = 0x30 | (retain ? 0x01 : 0x00);
    n += mqtt_len(buf + n, v);
    memcpy(buf + n, var, v);
    return n + v;
}
```

QoS 0 means the broker sends no acknowledgement. **You cannot tell from the device whether a fix was accepted.**

A rejected signature looks exactly like a successful publish. Section 10 explains how to check.

The `online` status publish needs QoS 1 and retain, which adds a two-byte packet

id after the topic and a PUBACK to wait for. If you would rather avoid that,

QoS 0 with retain set is acceptable for the status topic — the retain flag is

what matters, and the Last Will in the CONNECT is registered at QoS 1 regardless.

PINGREQ

```
const uint8_t PINGREQ[2] = {0xC0, 0x00};    // reply: 0xD0 0x00
```

Send it when 60 seconds have passed with nothing else sent. At the 5 second fix

interval you will rarely need it while driving, but you will while parked

between trips — and a missed keepalive is how the broker decides you are gone

and fires your Last Will.

Sending through the modem

```
AT+CIPSTART="TCP","broker.emqx.io","1883"    -> CONNECT OK
AT+CIPSEND=<exact byte count>                -> '>' prompt, then send raw bytes
```

Use the form with an explicit length. MQTT packets contain binary bytes

including nulls, so the Ctrl-Z terminated form is not safe here.

Check your SIM900A AT manual for the receive path — depending on the

`AT+CIPRXGET` mode, incoming bytes either arrive unsolicited as `+IPD` or must

be polled. You need it only for CONNACK and PINGRESP, but you do need it: a

connection that silently died looks identical to one that is working when you

never read from it.

7. Signing

ESP32 has SHA-256 in hardware. Use `mbedtls`, which is already in ESP-IDF and in

the Arduino core. Do not implement HMAC yourself.

```
#include "mbedtls/md.h"

// out_hex must be at least 65 bytes.
void hmac_sha256_hex(const char *key, const char *msg, char *out_hex) {
    unsigned char mac[32];
    mbedtls_md_context_t ctx;
    const mbedtls_md_info_t *info =
        mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);

    mbedtls_md_init(&ctx);
    mbedtls_md_setup(&ctx, info, 1);          // 1 = HMAC mode
    mbedtls_md_hmac_starts(&ctx, (const unsigned char *)key, strlen(key));
    mbedtls_md_hmac_update(&ctx, (const unsigned char *)msg, strlen(msg));
    mbedtls_md_hmac_finish(&ctx, mac);
    mbedtls_md_free(&ctx);

    for (int i = 0; i < 32; i++) sprintf(out_hex + i * 2, "%02x", mac[i]);
    out_hex[64] = '\0';
}
```

Lower case hex. The server compares in constant time and accepts either case, but keep it lower case to match the reference vector.

Building and signing the message

```
char body[256], sig[65], msg[400];

// hdg is null when the GPS has no course, never 0 -- zero means due north and
// would point every parked bus the same wrong way on the map.
if (have_heading) {
    snprintf(body, sizeof body,
        "{ \"b\": \"%s\", \"d\": \"%s\", \"c\": %lu, \"lat\": %.6f, \"lng\": %.6f, \"spd\": %.2f, \"hdg\": %.1f }",
        BUS_ID, DEVICE_ID, (unsigned long)counter,
        lat, lng, speed_mps, heading);
} else {
    snprintf(body, sizeof body,
        "{ \"b\": \"%s\", \"d\": \"%s\", \"c\": %lu, \"lat\": %.6f, \"lng\": %.6f, \"spd\": %.2f, \"hdg\": null }",
        BUS_ID, DEVICE_ID, (unsigned long)counter, lat, lng, speed_mps);
}

hmac_sha256_hex(DEVICE_SECRET, body, sig);
sprintf(msg, sizeof msg, "%s.%s", sig, body);
// publish msg -- and note that `body` was not rebuilt in between
```

Key order matters only in the sense that the signature covers whatever you produce; the server does not care about ordering because it verifies the raw bytes before parsing. What breaks it is building the string twice.

8. The counter

The counter `c` must be strictly greater than the previous accepted value, forever, per device. It stops someone recording a valid message and replaying it later.

The recommended scheme needs no flash writes at all:

```
counter = gps_unix_seconds();    // monotonic, survives reboot for free
```

Take it from the GPS time once a fix is available, then increment locally on each publish. GPS time only moves forward, so a reboot resumes above the old value automatically.

If there is no usable GPS clock at boot, fall back to NVS:

```
nvs_handle_t h;
nvs_open("bus", NVS_READWRITE, &h);
uint32_t c = 0;
nvs_get_u32(h, "counter", &c);
c += 100; // cover increments not yet written
nvs_set_u32(h, "counter", c);
nvs_commit(h);
nvs_close(h);
```

Bumping by 100 on boot covers increments that were never flushed. Do **not** write NVS on every publish: 7200 writes a day is unnecessary wear when the GPS clock gives you the same guarantee for free.

If the counter goes backwards, every message is rejected until it catches up.

Because MQTT QoS 0 has no reply, the unit looks perfectly healthy while publishing into a void. This is the single most likely cause of "the firmware works but the bus never appears".

9. Things that will bite

Do not buffer offline fixes. The old spec told you to hold them in a ring buffer and flush on reconnect. That is now wrong. A position carries no timestamp — the server stamps arrival time itself — so a fix flushed three minutes later is indistinguishable from a live one and teleports the bus across the map. Drop what you cannot send.

Do not use a short or obvious client id. MQTT requires client ids to be unique per broker, and **a second client presenting the same one disconnects the first.**

The two then fight in a reconnect loop, each kicking the other off. The symptom is a unit that connects and drops every few seconds for no reason visible in your code.

On a private broker `esp32-01` would be fine. On a shared public one it is not: somebody else will eventually pick the same string. Prefixing with the topic root, which is unguessable, makes a collision effectively impossible:

`cbt7f3c9e21b-esp32-01`.

This is not hypothetical. The server had a fixed id of `bustracker-processor` and hit exactly this, logging "Unspecified error" disconnects seven times in thirty seconds until it was namespaced.

Do not reconnect per fix. MQTT is a session. Connect once and hold it, sending PINGREQ to keep it alive. Reconnecting every 5 seconds multiplies data

and power use and throws away the Last Will that tells students the bus dropped off.

Re-publish `online` **after every reconnect**. A new session does not restore your previous retained status message if the Last Will fired in between.

Speed is metres per second. NMEA gives knots. Multiply by 0.514444.

Heading is `null`, **not** `0`, **when unknown**. GPS course is unreliable below walking pace, so expect to send null often while parked.

Six decimal places is plenty for latitude and longitude — about 0.1 m.

A Jio SIM will not work. Jio is VoLTE-only with no 2G network at all. Use Airtel, Vi or BSNL.

10. Testing

Because QoS 0 gives no acknowledgement, verify by watching the broker rather than the device. On a laptop:

```
mosquitto_sub -h broker.emqx.io -t 'cbt7f3c9e21b/bus/+/#' -v
```

Leave that running while you bring the unit up.

Check your signing first

Before flashing anything, make your HMAC agree with this reference vector.

```
body:    {"b":"bus1","d":"esp32-01","c":1,"lat":11.3185,"lng":75.9379,"spd":6.4,"hdg":312}
secret:  YOUR_SECRET_HERE
sig:     d0d9d4c131e5b25c618be70720948ae790b564804543d6f8a743a88716567e39
```

If your firmware produces anything else for that exact body and secret, the bug is in your HMAC or your string building, and no amount of network debugging will help. Reproduce it on your own machine with:

```
BODY='{ "b": "bus1", "d": "esp32-01", "c": 1, "lat": 11.3185, '
BODY=$BODY'"lng":75.9379,"spd":6.4,"hdg":312}'
printf '%s' "$BODY" | openssl dgst -sha256 -hmac "YOUR_SECRET_HERE" -hex
```

Publish a message by hand

This proves the topic, framing and broker are right, with no firmware involved:

```
mosquitto_pub -h broker.emqx.io -t 'cbt7f3c9e21b/bus/bus1/gps' -m "$SIG.$BODY"
```

Bench checklist

1. Unit connects; `online` appears on the status topic.
2. A signed publish appears on the gps topic, correctly framed.
3. Ask the server side to confirm it was **accepted** — it should appear on `cbt7f3c9e21b/live/buses`. A message on the gps topic only proves you

published, not that the signature passed.

4. Corrupt one byte of the signature. The server logs `bad signature` and the bus does not move.
 5. Power the unit off. `offline` appears on the status topic within about 90 seconds. This proves the Last Will is registered.
 6. Power-cycle. It resumes with no manual step and the counter does not restart.
 7. Pull the antenna, then reconnect it. It recovers without a reboot loop.
 8. Run 30 minutes. Memory stable, one connection held, no reconnect storm.
- Steps 3 and 5 are the ones people skip and then spend a day debugging.

11. Per-unit configuration

Each unit needs three values. They are issued separately and are not in this document.

Value	Example	Notes
Bus id	<code>bus1</code>	Determines the topic
Device id	<code>esp32-01</code>	Goes in the <code>d</code> field
Secret	32 hex characters	Signs the body, never transmitted

Store the secret in NVS, not in a source file that ends up in a repository. A device is bound to exactly one bus on the server, and the binding is checked in both directions: the topic you publish on and the `b` field inside the signed body must both match.

The six units are `esp32-01` through `esp32-06`, mapping to `bus1`, `bus2`, `bus3`, `bus4`, `ac_mbh` and `ac_lh`.

12. Data budget

About 200 bytes per message once MQTT and TCP overhead are counted: roughly 100 of payload, 65 of signature, the rest framing.

Interval	Per bus per day	Per bus per month	All six per month
5 s	1.6 MB	48 MB	0.3 GB
10 s	0.8 MB	24 MB	0.15 GB

A 500 MB monthly SIM is comfortable. This is roughly half what the HTTP version used, because MQTT does not resend method, path and headers on every fix.

Do not go slower than 5 seconds without telling the server team. Stop-arrival detection needs at least two fixes inside a 40 m radius, and at 25 km/h a bus crosses that in about 11 seconds.

13. Questions to raise before starting

1. Does PPP come up on your SIM900A revision? That decides Route A or Route B, and it is worth half a day to find out before committing.
2. Is the endpoint currently hard-coded or in NVS? If hard-coded, all six units need reflashing rather than reconfiguring.
3. Does the current firmware buffer offline fixes? If so, that code has to come out, not just be ported.

Questions come to us rather than being guessed at — a wrong assumption here costs a site visit to six buses.